The MeTTaKernel Appendix on Interactive Theorem Proving

A runnable, multi-system oracle curriculum — sufficient to focus a Lean-grade DTT + HOTG kernel

Abstract

This appendix is the ground-truth oracle for the MeTTa-hosted proof/inference kernel (see `papers/MettaKernel.tex`). It is a graded, *runnable* curriculum in four real checkers — **Coq/Rocq** and **Lean 4** (dependent type theory), **Megalodon** (higher-order Tarski–Grothendieck set theory), and **HOL4/CakeML** (verified programs). It is built to be *sufficient*: it exercises every feature a faithful kernel must implement *and* the proof-construction layers real users rely on. Every rung carries *positives* (theorems that check) and *negatives* (mistakes the checker catches). The single gate is `run_all.sh`: at the time of writing it reports **282 theorems across 56 files, 16 negatives caught, 0 failures**, plus an assumption ledger and a proved/tested/executed breakdown.

1 Purpose and method

A small trusted kernel is only as good as the behaviours it is focused against. This corpus is that focusing set. The discipline is strict: a positive must be *kernel-checked*; a negative must be *rejected by the real checker* (never asserted); "a test passed" is never recorded as "a theorem proved" (§8); and no binder-free or "representative" toy is allowed to stand in for the real feature.

Reproducing. `bash Mettapedia/MettaKernel/Curriculum/run_all.sh`.

2 The four checkers and what each pins

Checker	Foundation	What it pins for the kernel
Coq/Rocq 9.1	CIC (DTT)	terms/types, definitional conversion, i ductive families, the elimination restri tion, universe polymorphism, type class
Lean 4	CIC (DTT)	universes/`imax`, recursors, well-found recursion, quotients, proof irrelevance, the trusted-axiom base, and the ta tic/metaprogramming layer
Megalodon	HOTG	sets, extensionality, $\in$-induction, choic Tarski–Grothendieck universes, Replac ment/Separation, ω
HOL4 / CakeML	classical HO logic (Church STT) + LCF; verified compilation	a foundations ladder (HOL01–06, in the LCF primitive-rule kernel) *and* proc about programs / verified execution

3 Foundations ladders

Coq — the ICL spine (Smolka & Brown), Ch. 1–12, self-contained vanilla Rocq, negatives inline via `Fail`: types/props/equality/induction (01–04), the *elimination restriction* (05), sum/sigma + decidability (06), inductive predicates (07), lists (08), syntactic unification (09), natural deduction + Hilbert (10), Boolean semantics + tableau closure (11), Gentzen LJ (12). **Plus the metatheory I had skipped:** ND **soundness** (13); the classical **Kalmár completeness** theorem — every Boolean tautology is derivable (14, with weakening, the classical case-split, Kalmár's lemma, and atom elimination all carried out); and a **verified validity decision procedure** (15) — exhaustive valuation enumeration (the semantic tableau closing every branch), with soundness *and* completeness w.r.t. the semantics proved.

Lean — the DTT mirror (vanilla Lean 4): basics/induction/props/equality/ inductive types/Σ (01–06), then universes + explicit `Nat.rec` + definitional conversion + well-founded recursion (07).

Megalodon — HOTG (Egal mode): props/tactics/sets (01–03), extensionality + $\in$-induction + no-self-membership (04), iterated Tarski–Grothendieck closure (05), and — under the Egal preamble (loaded via a `.pre` sidecar) — unordered pairs/binary union/**choice via** ε/ω (06) and **Replacement/Separation/`ordsucc` with a genuine `nat_ind`** proof (07).

HOL — classical higher-order logic / LCF (HOL4): a fourth, distinct foundation. **HOL01** logic+tactics (`rw`/`simp`/`metis_tac`/`decide_tac`; forward + backward), **HOL02** induction (`Induct`/`Cases_on`), **HOL03** `Definition`/`Datatype`/`Inductive`, **HOL04** higher-order quantification + the Hilbert choice operator `@`/`SELECT`, **HOL05** classical reasoning, and **HOL06 the LCF kernel** — theorems built by the primitive inference rules (`REFL`/`ASSUME`/`DISCH`/`BETA_CONV`), the trusted core a MeTTa-hosted HOL kernel mirrors. Built into `.hol/objs`; negatives (a type error; an unprovable goal) are expected-fail `Holmake` builds.

HOL vs DTT vs HOTG (named exactly). The curriculum spans *three distinct foundation families*: **DTT** (Coq, Lean — the Calculus of Inductive Constructions); **HOTG** (Megalodon — higher-order Tarski–Grothendieck *set* theory); and **HOL** (HOL4 — classical Church *simple type theory* + Hilbert choice on an LCF kernel). HOL is not a weaker Megalodon: HOTG is HO *set* theory, HOL is HO *type* theory — different primitives, different kernels.

4 Proof construction: how real proofs are actually built

A kernel that checks terms is not enough; real developments are written through layers the curriculum now pins (Lean rungs 08–12, Coq `FEAT01`):

- **Type classes + instance resolution** (Lean 08; Coq `FEAT01`): dispatch by type, law-carrying classes, inheritance/derived instances; the negative is a failed instance synthesis.
- **Functor/Monad + `do`** (Lean 09): the monad laws proved, a user-defined `Monad` instance, `do` desugaring to `>>=`.
- **Tactic-mode** proving (Lean 10; Coq Ltac in `FEAT01`): `intro`/`apply`/`exact`/`cases`/`induction`/`rw`/`simp`/`ome`
- `conv` (Lean 11): navigate to a sub-term and rewrite there.
- **Metaprogramming** (Lean 12): `macro`, `syntax`+`macro_rules`, a custom tactic, and a term `elab`orator producing an `Expr`.

5 Kernel adequacy: what a faithful Lean kernel must implement

A kernel built only to the core DTT skeleton is *insufficient* — it fails on the first real proof that uses any of these. The curriculum exercises each (Lean rungs 13–15), and the trusted axioms are surfaced by `#print axioms` and recorded in the ledger:

Feature (rung)	Why a kernel needs it
Quotients (13)	`Quot`/`Quot.mk`/`Quot.lift` (definitional computation) + `Quot.sound`/`Quot.ind`. Mathlib's `Int`, `Rat`, quotient groups are quotients — a quotient-less kernel is dead on arrival.
Proof irrelevance (14)	any two proofs of a `Prop` are *definitionally* equal (Lean's kernel has this; Coq's does not).
η**-conversion** (14)	for functions *and* structures.
Impredicative `Prop` / `imax` (15)	`imax u 0 = 0`: a $\forall$ landing in `Prop` is a `Prop` regardless of the domain's universe.
Axioms (15)	`propext`, `Classical.choice` (and `funext` *derived* from `Quot`) — exercised and ledgered.

6 Program verification (the 4th pillar)

- **CoqPLF**: `Imp` big-step semantics; **Hoare logic** (rules proved sound against the semantics + a worked triple); type safety via **progress + preservation** for a typed expression language (`PV03_types_safety.v`, the labeled binder-free prelude); and the **real STLC** (`PV04_stlc.v` + `PV05_stlc_preservation.v`) with λ, substitution (SF-style, capture-permitting but sound for the closed values it is applied to), the substitution lemma, **progress AND preservation**.
- **Lean**: verified `reverse`/invariants/proof-carrying `Subtype`/type-indexed `Vec`.
- **MeTTaM1**: wires into the existing `metta-ref` development (a verified MeTTa interpreter slice in HOL4/CakeML). `run_all` uses the fast `make check-coverage` gate; the heavier `make test-hol` (HOL proofs + SML/CakeML) and `make test-oracle` (32 CeTTa-normalized comparisons) were run and *pass*.
- **HOL4CakeML**: a tutorial-scale verified function (`dbl`, with `dbl_two` and `dbl_add` proved) *and* its **CakeML translation** — the CakeML translator emits verified CakeML for `dbl` with a correctness certificate. Both build green under `Holmake`. HOL4-verified content is additionally exercised through MeTTaM1.

7 How the checker stops you: the error gallery

The payload of a negative is the rejection (the kernel must reproduce the soundness behaviour).

Negative	Checker	Message (verbatim, abridged)
ill-typed application	Lean	`Application type mismatch`
non-definitional `rfl`	Lean	`Not a definitional eq`
non-exhaustive `match`	Lean	`Missing cases`
`sorry` (as error)	Lean	`declaration uses 'sorry'`
missing type-class instance	Lean	`failed to synthesize`
bad `Quot.lift` (no respect proof)	Lean	`type mismatch`
non-terminating recursion	Lean	termination checker error
project an $\exists$ witness to data	Coq	elimination restriction: `Fail`
non-guarded `Fixpoint`	Coq	termination checker: `Fail`
wrong proof term	Megalodon	`does not match the current claim`

8 Assumptions: proved vs. tested vs. executed

The verifier never conflates three honesty classes: **proved** (kernel-checked theorem), **tested** (a test ran and passed), **executed** (a CakeML/oracle artifact ran). It prints an assumption ledger: the only classical axiom in the DTT ladder is excluded-middle `XM` (Coq Ch. 2; note the Kalmár calculus of Ch. 14 is classical by its `nd_raa` rule); the Lean kernel-adequacy rung is ledgered by `#print axioms` (`propext`, `Classical.choice`, `Quot.sound`); the Megalodon choice/ω rungs rest on the Egal preamble's HOTG axioms; and the HOTG ladder's own axioms (extensionality, $\in$-induction, the Tarski–Grothendieck universe axioms) are listed. None is counted as constructive evidence.

9 Map: each rung → the MeTTa-kernel feature it pins

Rung	Kernel feature it focuses
Coq 01–03 / Lean 01–04	term & type representation; **definitional conversion** ($\beta\delta\iota\zeta\eta$)
Coq 04 / Lean 02	structural recursion and induction
Coq 05–06	the `Prop`/`Type` split + the **elimination restriction**
Coq 07, 10–12	inductive judgments + the **proof-rule layer**: ND/Hilbert, tableau closure, Gentzen LJ — as calculi with worked derivations (*not* Gentzen cut-elimination)
Coq 13–15	ND **soundness**, classical **Kalmár completeness** (every tautology derivable), and a verified validity **decision procedure** (semantic/tableau)
Coq 09	the **unifier** for matching/elaboration
Lean 05–07	inductive families, **recursors**, universes, well-founded recursion
Lean 08–12 / Coq FEAT01	**proof-construction**: type classes/instance resolution, monads, tactics, `conv`, **metaprogramming**
Lean 13–15	**kernel adequacy**: quotients, proof irrelevance, η, `imax`/impredicative `Prop`, the trusted axioms
Megalodon 03–07	the **HO-set side**: sets, extensionality, $\in$-induction, choice, TG universes, Replacement/Separation, ω
HOL 01–06	the **HOL/LCF side**: classical HO simple type theory, Hilbert choice, the LCF primitive-rule kernel
CoqPLF (Imp/Hoare/STLC)	**proofs about programs and execution** — the EvalSpec capstone in miniature
MeTTaM1	the kernel running against **real MeTTa (M1) execution**

The right-hand column is the bridge to a future `Curriculum/MeTTa/`: the same rungs, re-expressed for the MeTTa-hosted kernel and measured against the oracle above.